Lecture 2: Network flows

Dr. George Mertzios

george.mertzios@durham.ac.uk

Room 2066, MCS Building
Tel: 42429

Flow networks, flow, maximum flow

A common scenario:

- material is transfered in a network from a "source" to a "sink"
- source produces material at a steady rate, sink consumes at same rate

Examples:

- water through system of pipes
- current through electrical network
- data through network
- trucks from factory to warehouse

Flow networks, flow, maximum flow

A common scenario:

- material is transfered in a network from a "source" to a "sink"
- source produces material at a steady rate, sink consumes at same rate

Examples:

- water through system of pipes
- current through electrical network
- data through network
- trucks from factory to warehouse

Edges have given capacity:

- gallons per minute / trucks per hour / kB per second ...

Vertices (other than source/sink) are "junctions":

- material flows through them without collecting in them
- entering rate = exiting rate (flow conservation)

Some definitions

Maximum-flow problem:

- we wish to compute greatest possible rate of transportation from source to sink

Flow network:

- $G = (V, E)$ is directed graph
- two distinguished vertices: source s and sink t
- each edge $(u, v) \in E$ has non-negative capacity $c(u, v) \geq 0$
- if $(u, v) \notin E$, we assume $c(u, v) = 0$.

We assume that every vertex lies on some path from s to t:

- for each $v \in V$, there is a path $s \rightsquigarrow v \rightsquigarrow t$

$\Rightarrow$ G is connected $\Rightarrow$ $|E| \geq |V| - 1$

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies these three properties:

Definition (Flow constraints)

- Capacity constraint:
  For all $u, v \in V$, we require $f(u, v) \leq c(u, v)$
  Flow from one vertex to another must not exceed given capacity.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies these three properties:

Definition (Flow constraints)

- Capacity constraint:
  For all $u, v \in V$, we require $f(u, v) \leq c(u, v)$
  Flow from one vertex to another must not exceed given capacity.
- Skew symmetry:
  For all $u, v \in V$, we require $f(u, v) = -f(v, u)$
  Flow from vertex u to vertex v is negative of flow in reverse direction.

A flow in G is a real-valued function $f : V \times V \to \mathbb{R}$ that satisfies these three properties:

Definition (Flow constraints)

- Capacity constraint:
  For all $u, v \in V$, we require $f(u, v) \leq c(u, v)$
  Flow from one vertex to another must not exceed given capacity.
- Skew symmetry:
  For all $u, v \in V$, we require $f(u, v) = -f(v, u)$
  Flow from vertex u to vertex v is negative of flow in reverse direction.
- Flow conservation:
  For all $u \in V - \{s, t\}$, we require

$$\sum_{v \in V} f(u, v) = 0$$

  Total flow out of a vertex (other than source or sink) is 0.
  Can rewrite as: $\forall v \in V - \{s, t\} : \sum_{u \in V} f(u, v) = 0$,
  i.e. total flow into a vertex is 0.

$f(u, v)$ — can be positive, zero, or negative — is called flow from u to v

Definition (Total positive flow)

The **total positive flow entering** vertex **v** is

$$\sum_{\mathbf{u} \in V:\ f(u,v)>0} f(u, v)$$

Also, **total positive flow leaving** vertex **u** is

$$\sum_{\mathbf{v} \in V:\ f(u,v)>0} f(u, v)$$

Definition (Total net flow)

The **total net flow** at a vertex v is

$$\begin{aligned} & \text{total positive flow leaving } v \\ - \quad & \text{total positive flow entering } v \end{aligned}$$

Definition (Flow value)

The **value** of flow f is defined as the total flow leaving the source (and thus entering the sink):

$$|f| = \sum_{v \in V} f(s, v)$$

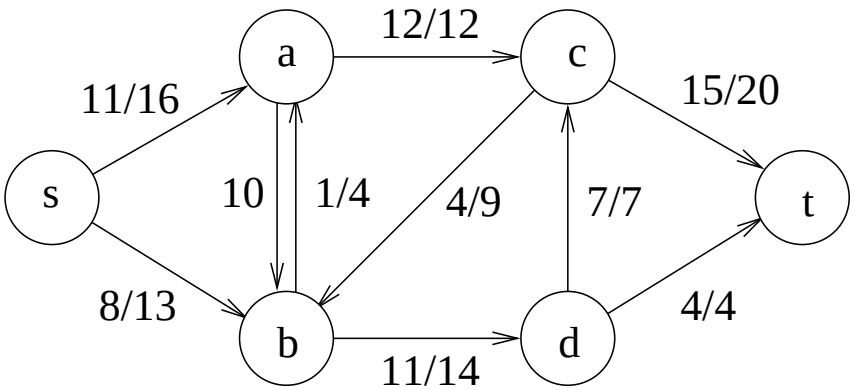
Note: $|\cdot|$ does not mean "absolute value" or "cardinality".

Definition (Flow value)

The **value** of flow f is defined as the total flow leaving the source (and thus entering the sink):

$$|f| = \sum_{v \in V} f(s, v)$$

Note: $|\cdot|$ does not mean "absolute value" or "cardinality".



Single numbers on the edges denote edge capacities.

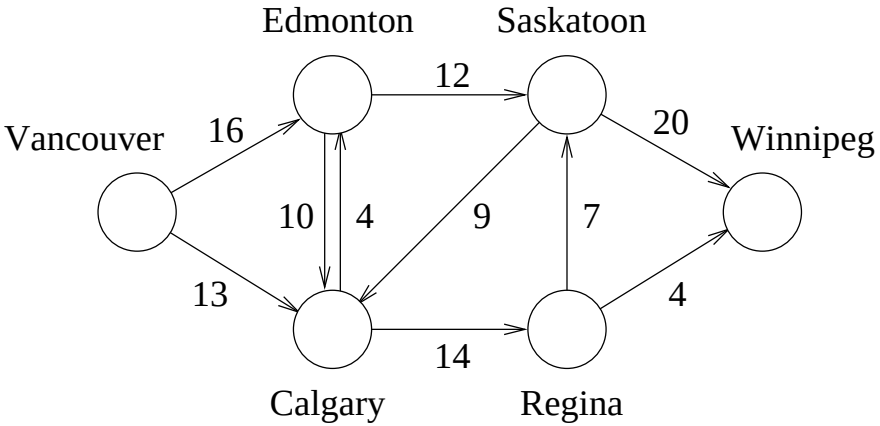
Pairs x/y denote $f(u, v)/c(u, v)$ (only labeled when $f(u, v) > 0$; if $f(u, v) \leq 0$, then only capacity is shown).

Figure shows flow f in G with value $|f| = 19$.

An example

The **Lucky Puck Company** produces hockey pucks:

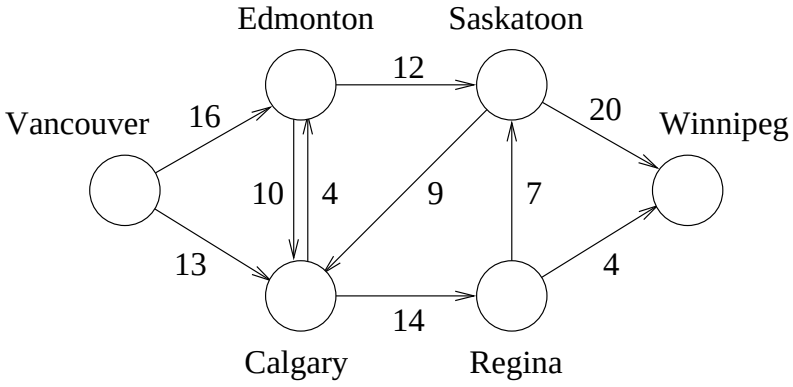
- it has factory (source) in Vancouver and warehouse (sink) in Winnipeg
- they lease space on trucks to ship pucks from factory to warehouse
- trucks have limited capacity
- trucks take specified routes (edges) between cities
- they can ship $\leq c(u, v)$ items per day from city u to city v
- goal is to find largest number p of items per day that can be shipped, and then to produce this amount each day
- no point in producing more pucks than what can be transported to warehouse



Seems appropriate to model "flow" of shipments with flow in network:

- have capacity constraint (items shipped per day from one city to another is limited)
- flow conservation must be obeyed (in steady state, rate at which pucks enter intermediate city must equal the rate at which they leave — otherwise, accumulation)

Cancellation



However, **one difference** between shipments and flows:

Suppose they ship 8 crates per day from Edmonton to Calgary, and 3 crates per day from Calgary to Edmonton.

Natural enough, but we can't represent these shipments directly by flows: **skew-symmetry is violated** [must have $f(u, v) = -f(v, u)$].

However, Lucky Puck's strategy is pointless anyway:

> why ship 8 crates from Edmonton to Calgary and 3 creates back, instead of just shipping 5 crates from Edmonton to Calgary?

In effect, 3 of the 8 crates from Edmonton to Calgary are **cancelled** by the 3 crates from Calgary to Edmonton.

We represent this with a flow: $f(Ed, Ca) = 5$ and $f(Ca, Ed) = -5$.

Technical tools

Implicit summation. Let $X, Y \subseteq V$. Then

$$f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f(x, y)$$

For example, flow-conservation

$$\sum_{v \in V} f(u, v) = 0 \quad \forall u \in V - \{s, t\}$$

can be written simply as

$$f(u, V) = 0 \quad \forall u \in V - \{s, t\}$$

Technical tools (continued)

Commonly occurring identities:

1. For all $X \subseteq V$, we have $f(X, X) = 0$
   Because each $f(u, v)$ and $f(v, u) = -f(u, v)$ cancel each other.

2. For all $X, Y \subseteq V$, we have $f(X, Y) = -f(Y, X)$
   Generalization of $f(X, X) = 0$, same reasoning.

3. For all $X, Y, Z \subseteq V$ with $X \cap Y = \emptyset$, we have

$$f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$$

$$f(Z, X \cup Y) = f(Z, X) + f(Z, Y)$$

   Split summation into two: one over X, one over Y.

The Ford-Fulkerson method

Three important ideas:

1. residual networks
2. augmenting paths
3. cuts

Method is iterative:

1. Start with $f(u, v) = 0$ for all $u, v \in V$
2. At each iteration, increase flow value by finding an augmenting path — a path from source to sink along which we can increase flow — and then augment flow along this path.
3. Repeat until no augmenting path can be found.

Residual networks

Idea: Residual network consists of edges that can admit more flow.

Formally: Consider vertices u and v. Amount of additional flow we can push from u to v before exceeding capacity $c(u, v)$ is residual capacity of (u, v):

$$c_f(u, v) = c(u, v) - f(u, v)$$

Residual networks

Idea: Residual network consists of edges that can admit more flow.

Formally: Consider vertices u and v. Amount of additional flow we can push from u to v before exceeding capacity $c(u, v)$ is residual capacity of (u, v):

$$c_f(u, v) = c(u, v) - f(u, v)$$

Note: When flow $f(u, v)$ is negative, then residual capacity $c_f(u, v)$ is greater than $c(u, v)$.

Interpretation:

- flow of $-x$ from u to v (i.e. $f(u, v) = -x < 0$)
- implies flow of x from v to u (i.e. $f(v, u) = x > 0$)
- can be canceled by pushing x units from u back to v
- can then push another $c(u, v)$ from u to v

$\Rightarrow$ we can push in total $c_f(u, v) = c(u, v) + x > c(u, v)$ from u to v

Residual networks

Given flow network $G = (V, E)$ and flow f, the residual network of G induced by f is $G_f = (V, E_f)$ with

$$E_f = \{(u, v) \in V \times V : \; c_f(u, v) > 0\}$$

i.e. each residual edge that can admit flow that is strictly positive.

$$G: \quad u \underset{1}{\overset{3/5}{\rightleftarrows}} v \qquad \Longrightarrow \qquad G_f: \quad u \underset{4}{\overset{2}{\rightleftarrows}} v$$

Note: $|E_f| \leq 2|E|$.

Lemma

Let $G = (V, E)$ be a flow network, f be a flow in G, G_f be the residual network of G induced by f, and let f' be a flow in G_f. Then the flow sum $f + f'$ with

$$(f + f')(u, v) = f(u, v) + f'(u, v)$$

is a flow in G with value $|f + f'| = |f| + |f'|$.

Proof. Must verify that skew symmetry, capacity constraints, and conservation are obeyed.

Skew symmetry:

$$\begin{aligned}
(f + f')(u, v) &= f(u, v) + f'(u, v) \\
&= -f(v, u) - f'(v, u) \\
&= -(f(v, u) + f'(v, u)) \\
&= -(f + f')(v, u)
\end{aligned}$$

Capacity constraint: Note f' is flow in G_f, so $f'(u,v) \leq c_f(u,v)$. Since by def. $c_f(u,v) = c(u,v) - f(u,v)$,

$$\begin{aligned}
(f+f')(u,v) &= f(u,v) + f'(u,v) \\
&\leq f(u,v) + c_f(u,v) \\
&= f(u,v) + (c(u,v) - f(u,v)) \\
&= c(u,v)
\end{aligned}$$

Capacity constraint: Note f' is flow in G_f, so $f'(u,v) \leq c_f(u,v)$. Since by def. $c_f(u,v) = c(u,v) - f(u,v)$,

$$\begin{aligned}
(f+f')(u,v) &= f(u,v) + f'(u,v) \\
&\leq f(u,v) + c_f(u,v) \\
&= f(u,v) + (c(u,v) - f(u,v)) \\
&= c(u,v)
\end{aligned}$$

Flow conservation: For all $u \in V - \{s,t\}$,

$$\begin{aligned}
\sum_{v \in V} (f+f')(u,v) &= \sum_{v \in V} (f(u,v) + f'(u,v)) \\
&= \sum_{v \in V} f(u,v) + \sum_{v \in V} f'(u,v) \\
&= 0 + 0 \\
&= 0
\end{aligned}$$

Finally, value:

$$
\begin{aligned}
|f + f'| &= \sum_{v \in V} (f + f')(s, v) \\
&= \sum_{v \in V} (f(s, v) + f'(s, v)) \\
&= \sum_{v \in V} f(s, v) + \sum_{v \in V} f'(s, v) \\
&= |f| + |f'|
\end{aligned}
$$

$\square$

Augmenting paths

Given flow network $G = (V, E)$ and flow f:

- an augmenting path P is a simple path in residual network G_f

Recall: each edge (u, v) in G_f admits some additional positive flow, obeying capacity constraint.

Flow value can be increased by

$$c_f(P) = \min_{(u,v) \in P} c_f(u, v)$$

Augmenting paths

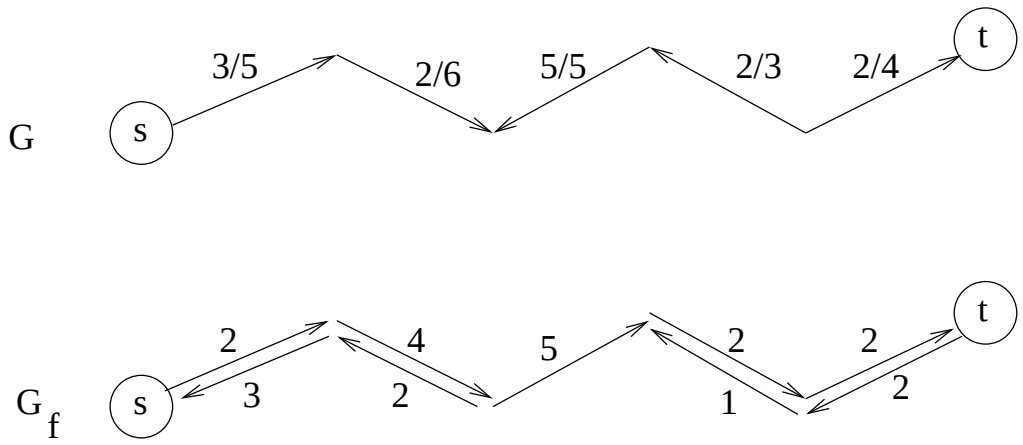
Given flow network $G = (V, E)$ and flow f:

- an augmenting path P is a simple path in residual network G_f

Recall: each edge (u, v) in G_f admits some additional positive flow, obeying capacity constraint.

Flow value can be increased by

$$c_f(P) = \min_{(u,v) \in P} c_f(u, v)$$



In this example: possible augmentation by at most ???

Augmenting paths

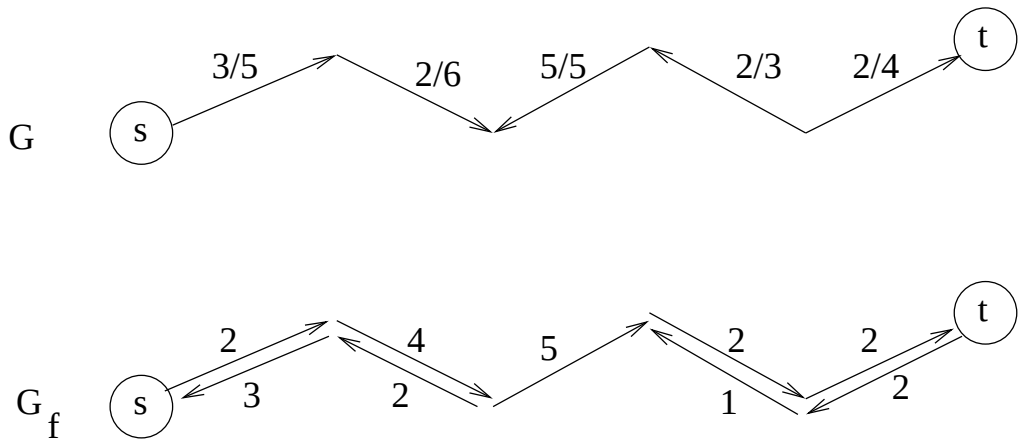
Given flow network $G = (V, E)$ and flow f:

- an augmenting path P is a simple path in residual network G_f

Recall: each edge (u, v) in G_f admits some additional positive flow, obeying capacity constraint.

Flow value can be increased by

$$c_f(P) = \min_{(u,v) \in P} c_f(u, v)$$



In this example: possible augmentation by at most 2

Lemma

Let $G = (V, E)$ be a flow network, f be a flow in G, and let P be an augmenting path in G_f. Define f_P by:

$$f_P(u,v) = \begin{cases} c_f(P) & \text{if } (u,v) \text{ is on } P \\ -c_f(P) & \text{if } (v,u) \text{ is on } P \\ 0 & \text{otherwise} \end{cases}$$

Then f_P is a flow in G_f with value $|f_P| = c_f(P) > 0$.

Proof: Exercise.

Lemma

Let $G = (V, E)$ be a flow network, f be a flow in G, and let P be an augmenting path in G_f. Define f_P by:

$$f_P(u,v) = \begin{cases} c_f(P) & \text{if } (u,v) \text{ is on } P \\ -c_f(P) & \text{if } (v,u) \text{ is on } P \\ 0 & \text{otherwise} \end{cases}$$

Then f_P is a flow in G_f with value $|f_P| = c_f(P) > 0$.

Proof: Exercise.

Corollary (Improve)

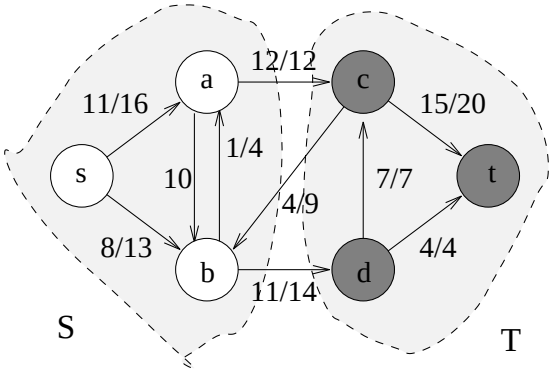
Let G, f, P, f_P be as above. Define $f' = f + f_P$. Then f' is a flow in G with value $|f'| = |f| + |f_P| > |f|$.

Proof: From last two lemmas.

Cuts of flow networks

A cut of a flow network $G = (V, E)$ is a partition of V into the sets S and $T = V - S$ such that $s \in S$ and $t \in T$.

If f is a flow in G, then $f(S, T)$ is the **net flow** across cut (S, T); its **capacity** is $c(S, T)$.



Flow: $f(S, T) = f(a, c) + f(b, c) + f(b, d) = 12 + (-4) + 11 = 19$
Note: can include negative flows.

Capacity: $c(S, T) = c(a, c) + c(b, d) = 12 + 14 = 26$
Note: only non-negative values (i.e. only edges from S to T).

A minimum cut is a cut with minimum capacity over all cuts.

Net flow across any cut is the same:

Lemma (Net flow)

Let f be a flow in a flow network G with source s and sink t, let (S, T) be a cut of G. Then the flow across (S, T) is $f(S, T) = |f|$.

Net flow across any cut is the same:

Lemma (Net flow)

Let f be a flow in a flow network G with source s and sink t, let (S, T) be a cut of G. Then the flow across (S, T) is $f(S, T) = |f|$.

Corollary (Upper cut)

The value of a flow f in a network G is upper-bounded by the capacity of any cut (S, T) in G.

Proof.

$$|f| = f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) \leq \sum_{u \in S} \sum_{v \in T} c(u, v) = c(S, T)$$

□

Net flow across any cut is the same:

Lemma (Net flow)

Let f be a flow in a flow network G with source s and sink t, let (S, T) be a cut of G. Then the flow across (S, T) is $f(S, T) = |f|$.

Corollary (Upper cut)

The value of a flow f in a network G is upper-bounded by the capacity of any cut (S, T) in G.

Proof.

$$|f| = f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) \leq \sum_{u \in S} \sum_{v \in T} c(u, v) = c(S, T)$$

□

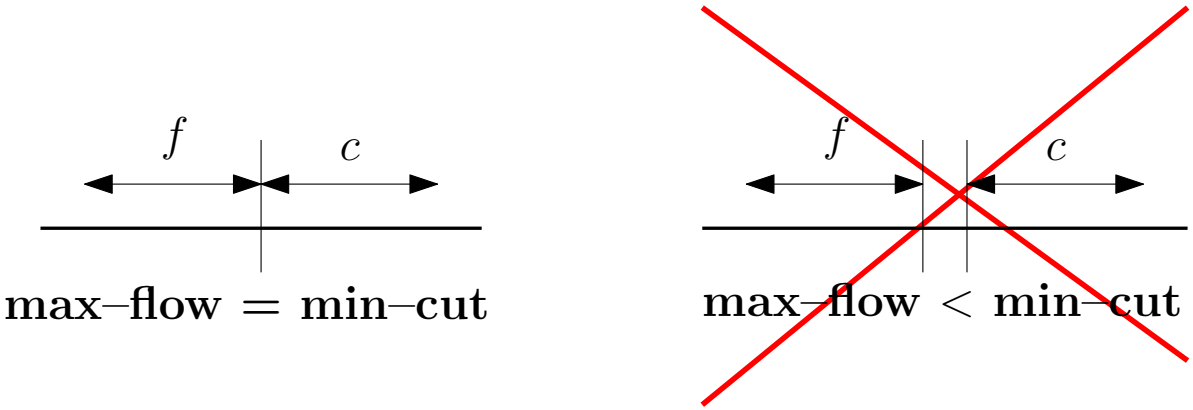
Therefore: **max–flow** $\leq$ **min–cut**

Improving last corollary, the next theorem shows equality:

Theorem (Max-flow min-cut)

If f is a flow in a flow network $G = (V, E)$ with source s and sink t, then the following conditions are equivalent:

1. *f is a maximum flow in G.*
2. *The residual network G_f contains no augmenting path.*
3. *$|f| = c(S, T)$ for some cut (S, T) of G.*



Basic Ford-Fulkerson algorithm

Input: G, s, t

1: **for** each edge $(u, v) \in E$ **do**
2: $\quad f(u, v) \leftarrow 0$
3: $\quad f(v, u) \leftarrow 0$
4: **end for**
5: **while** there exists a path $P = s \rightsquigarrow t$ in the residual network G_f **do** {i.e. augmenting path in G}
6: $\quad c_f(P) \leftarrow \min\{c_f(u, v) : (u, v) \in P\}$
7: $\quad$ **for** each edge $(u, v) \in P$ **do**
8: $\quad\quad f(u, v) \leftarrow f(u, v) + c_f(P)$
9: $\quad\quad f(v, u) \leftarrow -f(u, v)$
10: $\quad$ **end for**
11: **end while**

Question: How to find an augmenting path P of G in line 5?

Basic Ford-Fulkerson algorithm

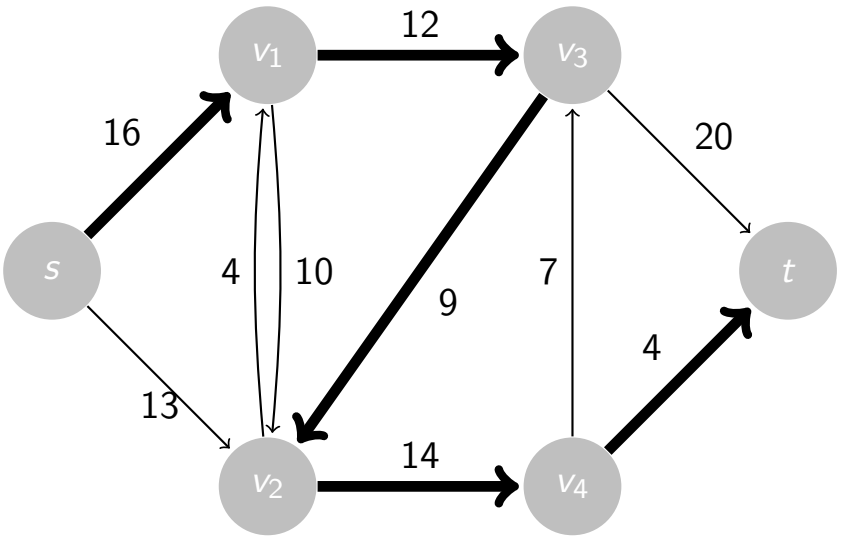
Input: G, s, t

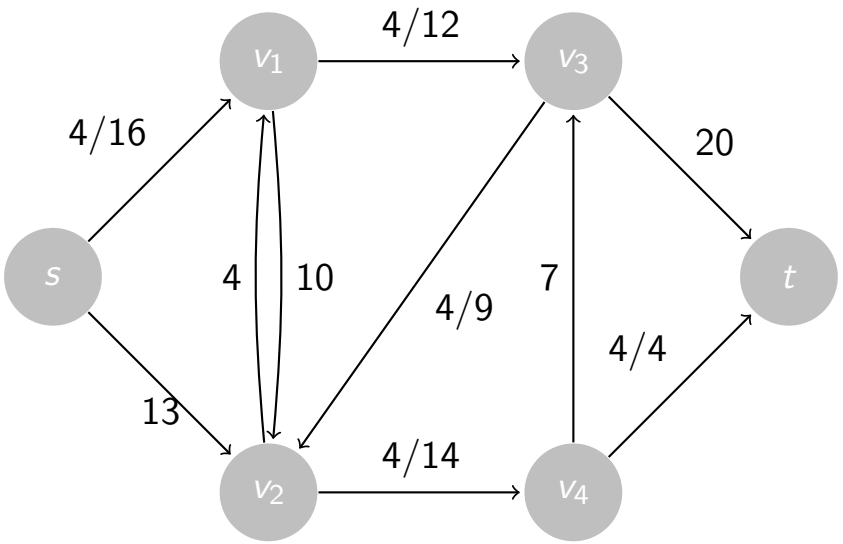
1: **for** each edge $(u, v) \in E$ **do**
2: $\quad f(u, v) \leftarrow 0$
3: $\quad f(v, u) \leftarrow 0$
4: **end for**
5: **while** there exists a path $P = s \rightsquigarrow t$ in the residual network G_f **do** {i.e. augmenting path in G}
6: $\quad c_f(P) \leftarrow \min\{c_f(u, v) : (u, v) \in P\}$
7: $\quad$ **for** each edge $(u, v) \in P$ **do**
8: $\quad\quad f(u, v) \leftarrow f(u, v) + c_f(P)$
9: $\quad\quad f(v, u) \leftarrow -f(u, v)$
10: $\quad$ **end for**
11: **end while**

Question: How to find an augmenting path P of G in line 5?

Answer: Not specified by the algorithm !

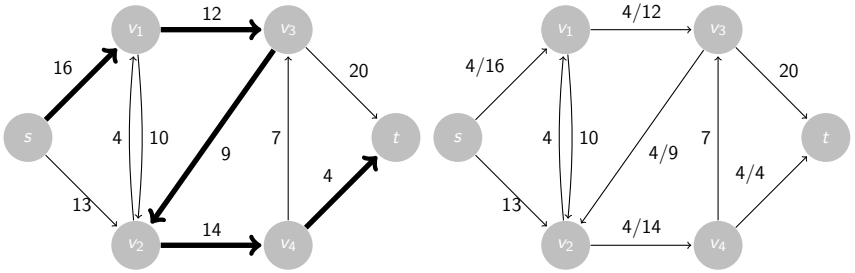
Example: step 1

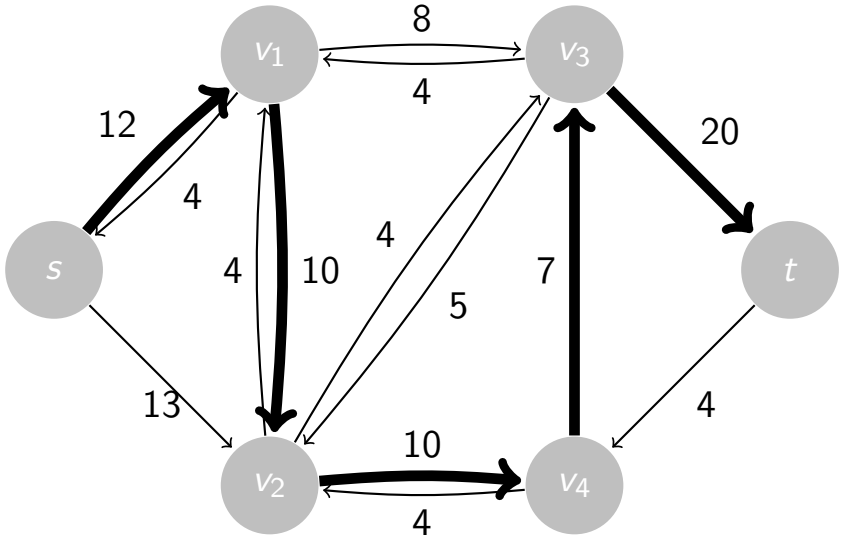


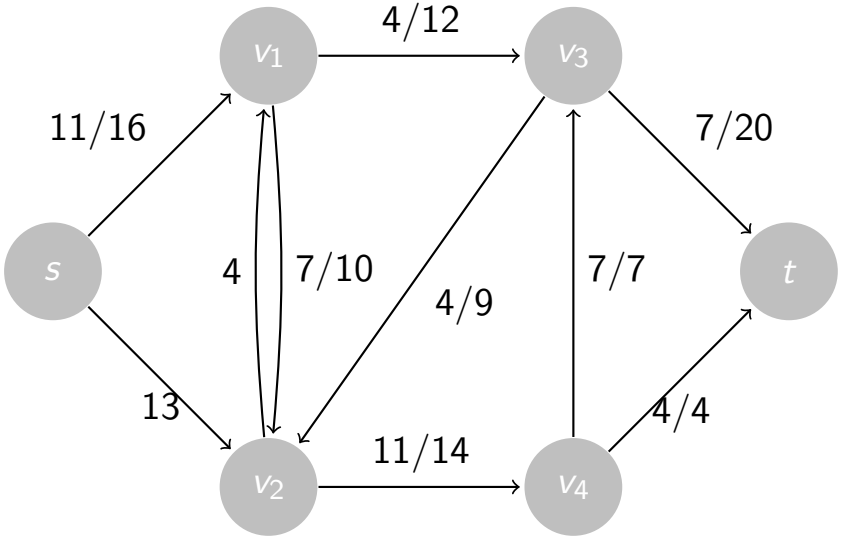


residual network on the left, resulting flow network on the right

Example: step 2

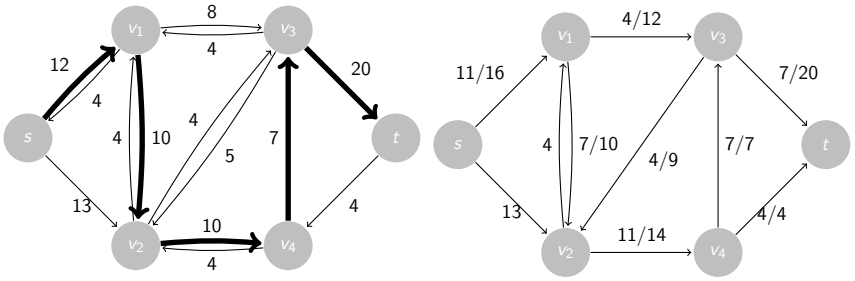


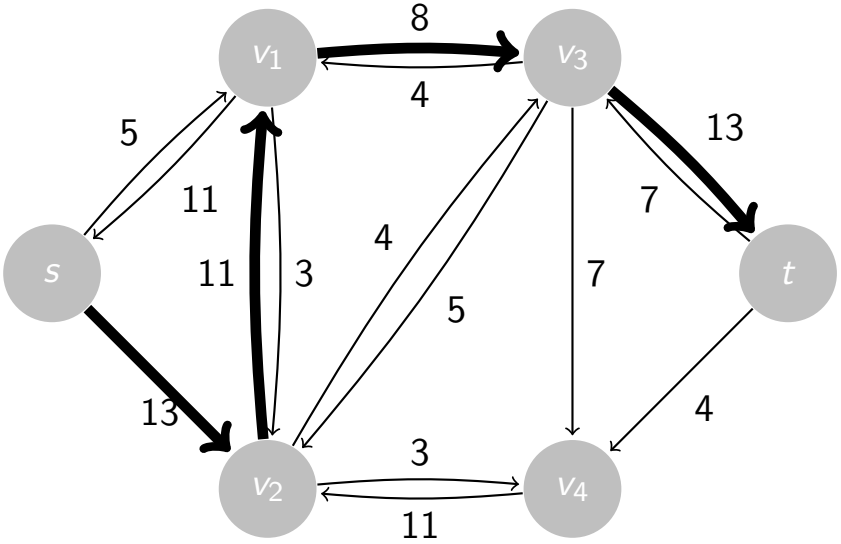


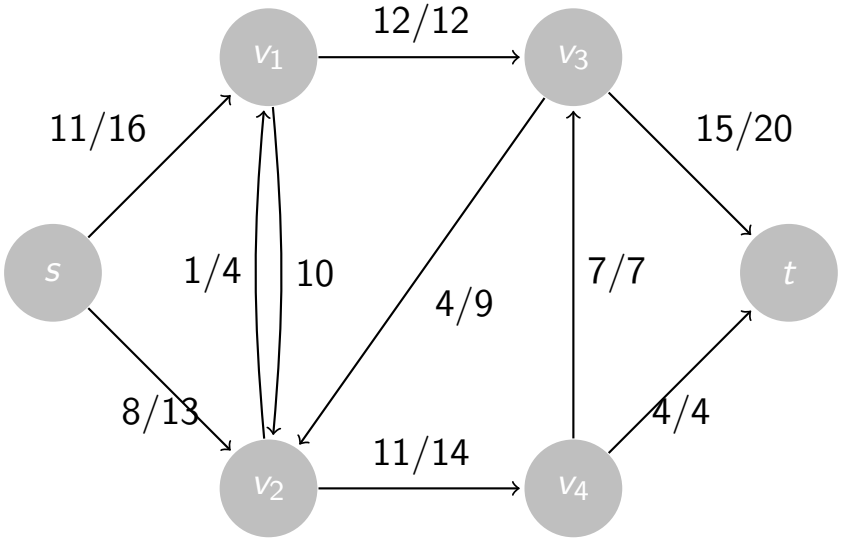


residual network on the left, resulting flow network on the right

Example: step 3

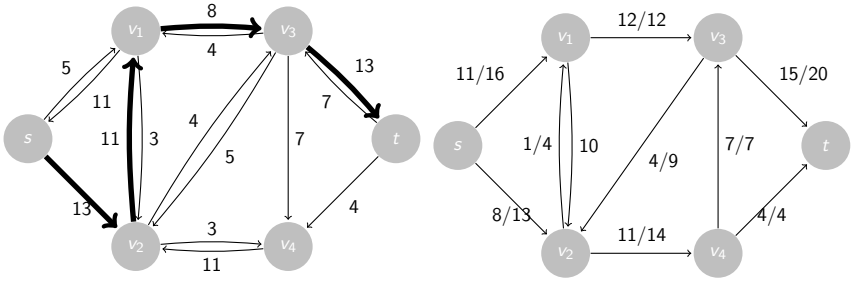


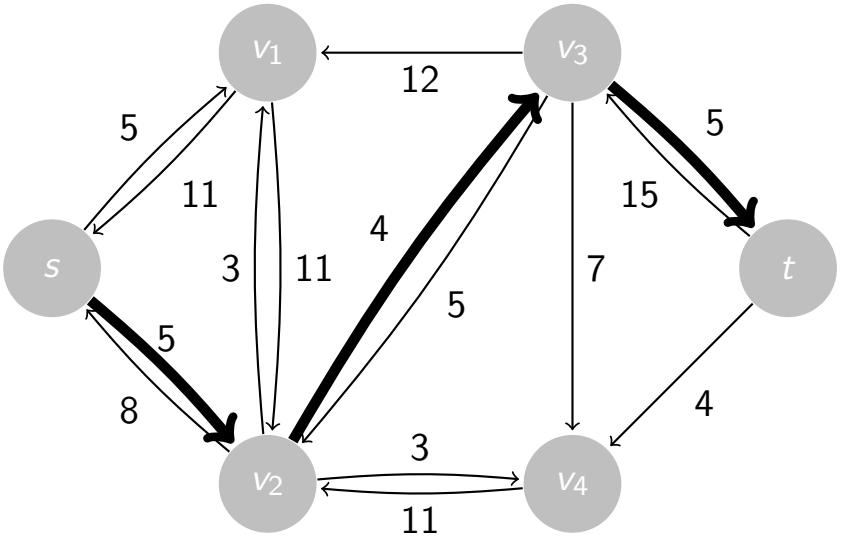


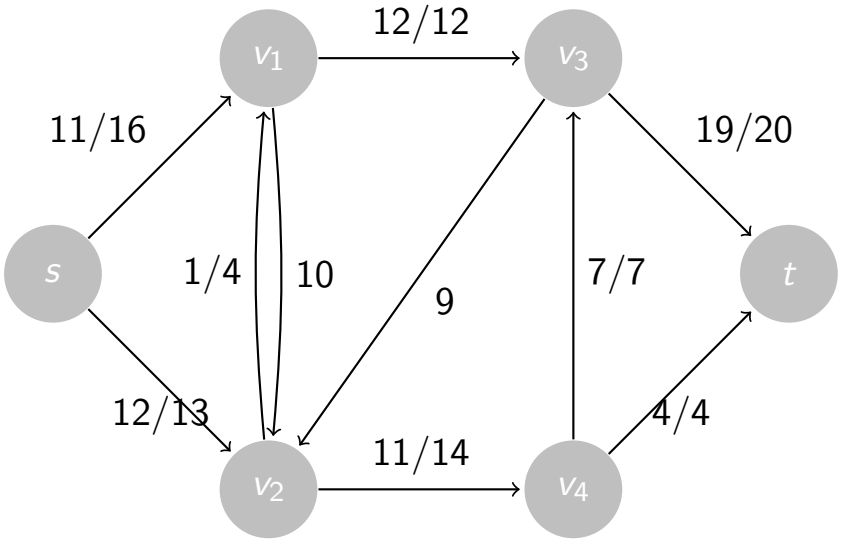


residual network on the left, resulting flow network on the right

Example: step 4

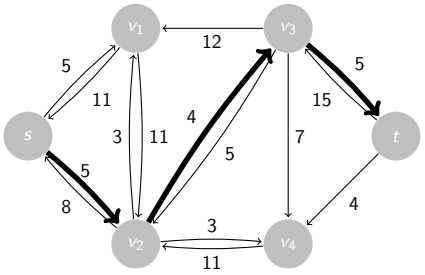


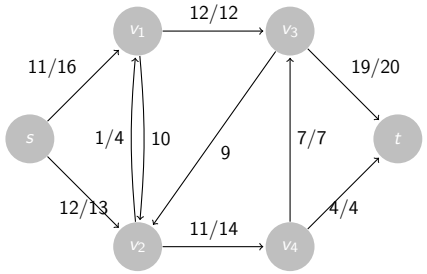


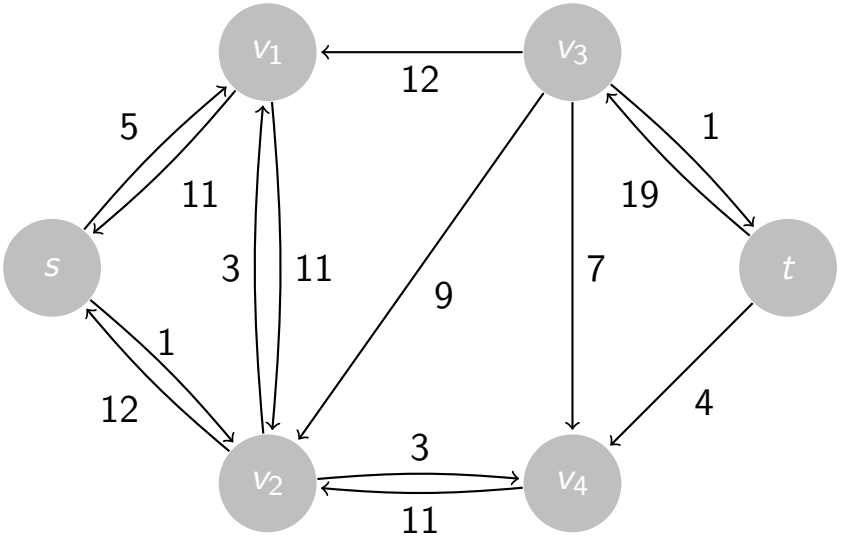


residual network on the left, resulting flow network on the right

Example: step 5







residual network; no $s - t$ path left $\Longrightarrow$ done!

Running time

It strongly depends on how augmenting paths in line 5 are determined.

If chosen poorly:

- value of flow increases with each iteration
- but possibly too slowly !
- in extreme cases it might never terminate,
  and it can even not converge to the value of maximum flow!
  (this can happen only if the capacities are irrational numbers)

Running time

It strongly depends on how augmenting paths in line 5 are determined.

If chosen poorly:

- value of flow increases with each iteration
- but possibly too slowly !
- in extreme cases it might never terminate,
  and it can even not converge to the value of maximum flow!
  (this can happen only if the capacities are irrational numbers)

In practice:

- capacities are integers

$\Rightarrow$ always terminates (each time increase flow value by ≥ 1)

- if all capacities are small, then efficient algorithm (polynomial time), otherwise...

A simple running time analysis

Assume **integral capacities**. Simple bound $O(E \cdot |f^*|)$ for running time when choosing paths **arbitrarily**, $|f^*|$ being the **value of maximum flow**:

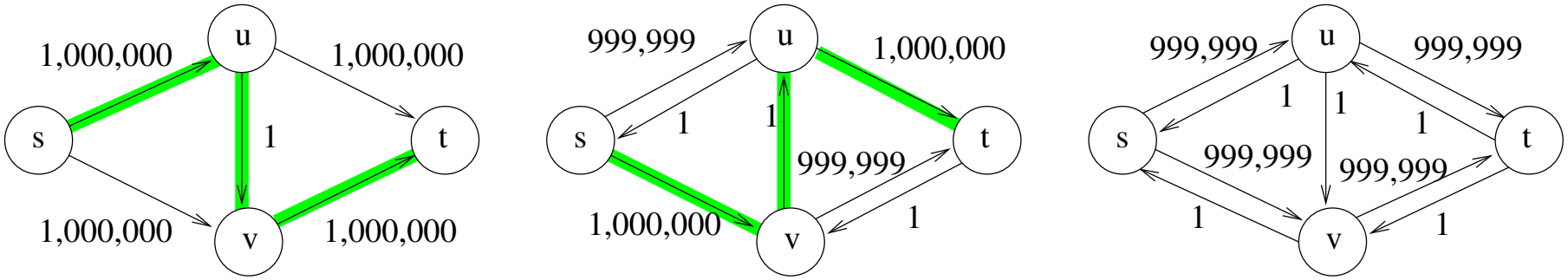
- **initialization** in lines 1–4 take $O(E)$
- **while** loop in lines 5–11 is executed at most $|f^*|$ times (value of flow increases by at least one unit in each iteration)
- in lines 6–10 (within the while loop) we need $O(V + E) = O(E)$ time (just graph traversing by DFS/BFS and then path traversing)

Input: G, s, t

1: **for** each edge $(u, v) \in E$ **do**
2: $\quad f(u, v) \leftarrow 0$
3: $\quad f(v, u) \leftarrow 0$
4: **end for**
5: **while** there exists a path $P = s \rightsquigarrow t$ in the residual network G_f **do** {i.e. augmenting path in G}
6: $\quad c_f(P) \leftarrow \min\{c_f(u, v) : (u, v) \in P\}$
7: $\quad$ **for** each edge $(u, v) \in P$ **do**
8: $\quad\quad f(u, v) \leftarrow f(u, v) + c_f(P)$
9: $\quad\quad f(v, u) \leftarrow -f(u, v)$
10: $\quad$ **end for**
11: **end while**

Problematic case:

Algorithm needs $\Omega(E|f^*|)$ time in worst case, because (u, v) is always chosen to be part of augmenting path. Here: $|f^*| = 2,000,000$



Important:

- the value of the maximum flow f^* can be arbitrarily large (might be even exponential on the size of the network!)

$\Rightarrow$ this algorithm has not polynomial running time in worst case

A polynomial algorithm

Edmonds-Karp algorithm:

- a special implementation of Ford-Fulkerson algorithm
- the augmenting path P is always chosen to be a shortest path from s to t (e.g. using BFS) in the residual network G_f
- regardless of the flow that fits in this path P

Running time:

- in total at most $O(V \cdot E)$ augmentations

$\Rightarrow$ running time $O(V \cdot E^2)$, since each augmentation needs $O(E)$ time (i.e. polynomial on the network size)

Edmonds-Karp algorithm: running time analysis

Lemma (Monotonicity of Edmonds-Karp algorithm)

For every vertex $v \in V - \{s, t\}$, the shortest-path distance $\delta_f(s, v)$ in the residual network G_f increases monotonically after each flow augmentation.

Proof idea.

Assume otherwise and derive a contradiction. □

Edmonds-Karp algorithm: running time analysis

Lemma (Monotonicity of Edmonds-Karp algorithm)

For every vertex $v \in V - \{s, t\}$, the shortest-path distance $\delta_f(s, v)$ in the residual network G_f increases monotonically after each flow augmentation.

Proof idea.

Assume otherwise and derive a contradiction. □

Lemma (Number of flow augmentations)

The algorithm performs at most $O(V \cdot E)$ flow augmentations.

Proof idea.

- Every augmentation has at least one "critical" edge.
- Every edge becomes at most $\frac{V}{2}$ times "critical" during the algorithm.
- there are $O(E)$ edges $\Rightarrow$ at most $O(V \cdot E)$ augmentations. □

Applications of the Maximum Flow problem

The Dancing Party problem:

- a group X of men and a group Y of women, all in a dancing party
- $x \in X$ is "matchable" with $y \in Y$ if they are both willing to dance with each other
- what is the largest set of pairs happily dancing at the same time?

Applications of the Maximum Flow problem

The Dancing Party problem:

- a group X of men and a group Y of women, all in a dancing party
- $x \in X$ is "matchable" with $y \in Y$ if they are both willing to dance with each other
- what is the largest set of pairs happily dancing at the same time?

Maximum Matching Problem

Let $G = (V, E)$ be an undirected graph. A matching in G is a set of edges of E with no common endpoints.

Applications of the Maximum Flow problem

The Dancing Party problem:

- a group X of men and a group Y of women, all in a dancing party
- $x \in X$ is "matchable" with $y \in Y$ if they are both willing to dance with each other
- what is the largest set of pairs happily dancing at the same time?

Maximum Matching Problem

Let $G = (V, E)$ be an undirected graph. A matching in G is a set of edges of E with no common endpoints.

- We "pair" neighboring vertices of V such that each vertex is "paired" with at most one other vertex of V.
- Special case: G is bipartite, i.e. V is partitioned into two sets X, Y such that every edge is between one $x \in X$ and one $y \in Y$.

Maximum Bipartite Matching

Many further applications, for example:

- X is a set of machines and Y a set of jobs
- $x \in X$ is connected $y \in Y$ if job y can be executed on machine x
- maximum matching $\Leftrightarrow$ maximum number of executed jobs

Maximum Bipartite Matching

Many further applications, for example:

- X is a set of machines and Y a set of jobs
- $x \in X$ is connected $y \in Y$ if job y can be executed on machine x
- maximum matching $\Leftrightarrow$ maximum number of executed jobs

We can simply solve this problem using Maximum Flows:

- let $G = (X, Y, E)$ be a bipartite graph
- we construct from G a flow network H such that every maximum flow in H can be immediately converted to a maximum matching in G

$\rightarrow$ then apply Ford-Fulkerson algorithm on H !

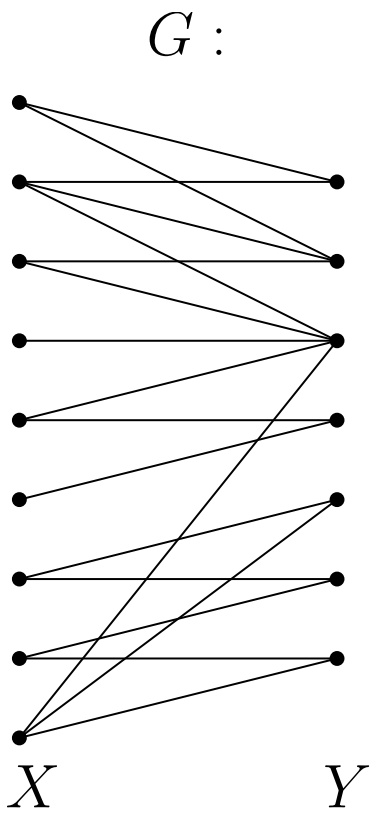
That is: we "reduce" the bipartite matching problem to the maximum flow problem (which we know how to solve)

Maximum Bipartite Matching

1. we include in H all vertices of G, plus a new source s and sink t
2. we add all edges of G and direct them all from X to Y

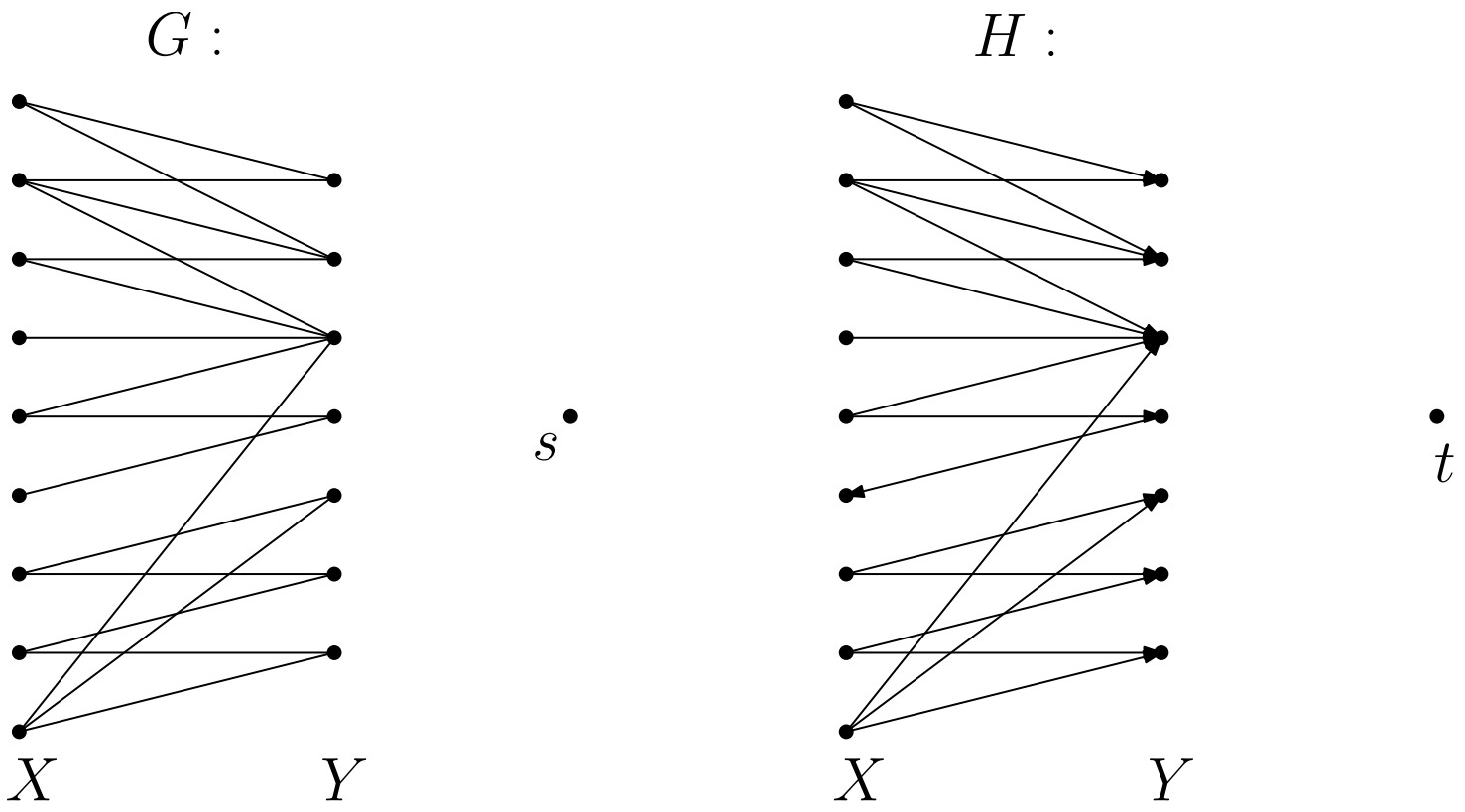
Maximum Bipartite Matching

1. we include in H all vertices of G, plus a new source s and sink t
2. we add all edges of G and direct them all from X to Y



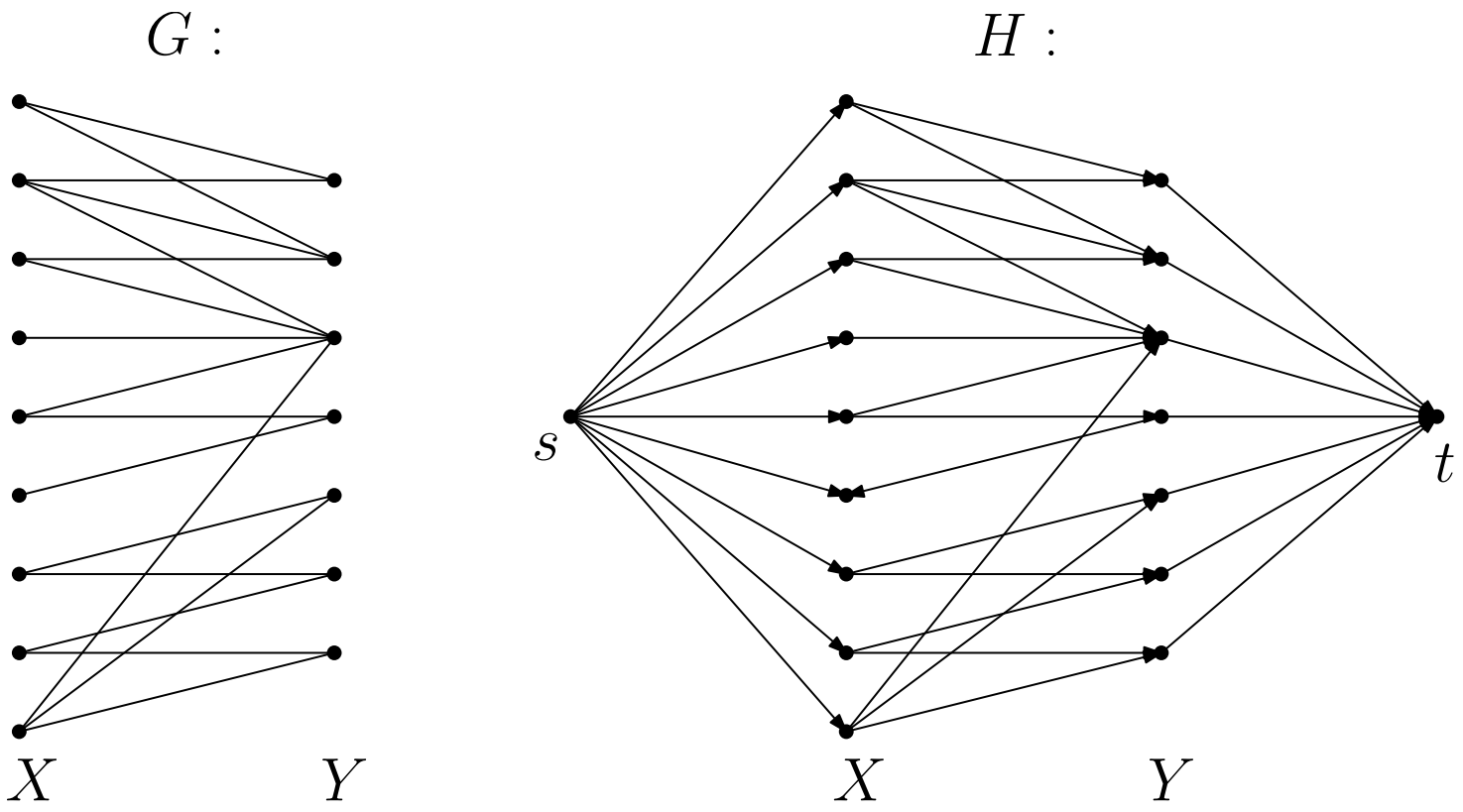
Maximum Bipartite Matching

1. we include in H all vertices of G, plus a new source s and sink t
2. we add all edges of G and direct them all from X to Y



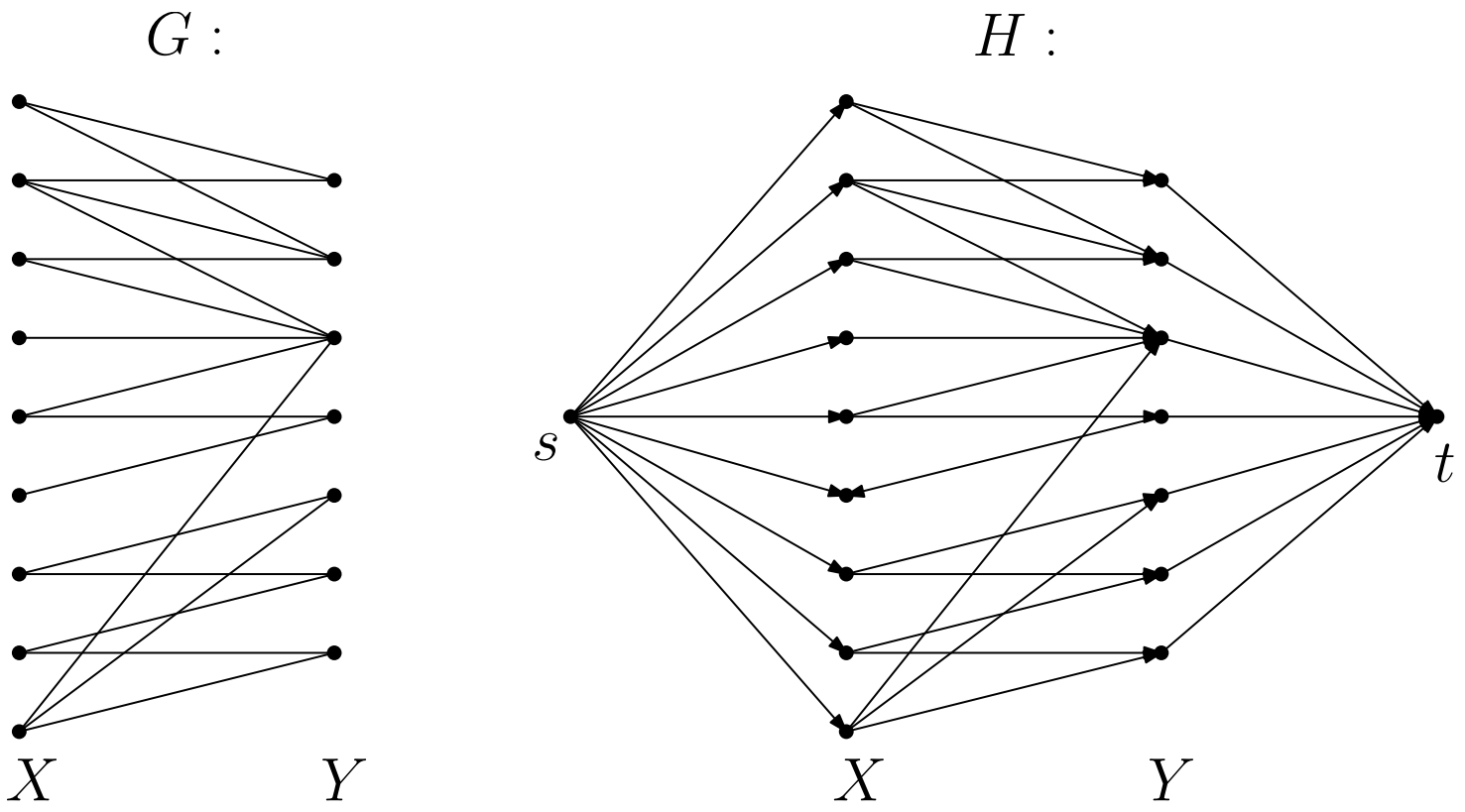
Maximum Bipartite Matching

1. we include in H all vertices of G, plus a new source s and sink t
2. we add all edges of G and direct them all from X to Y
3. we add a directed edge from s to all vertices in X, and from all vertices of Y to t



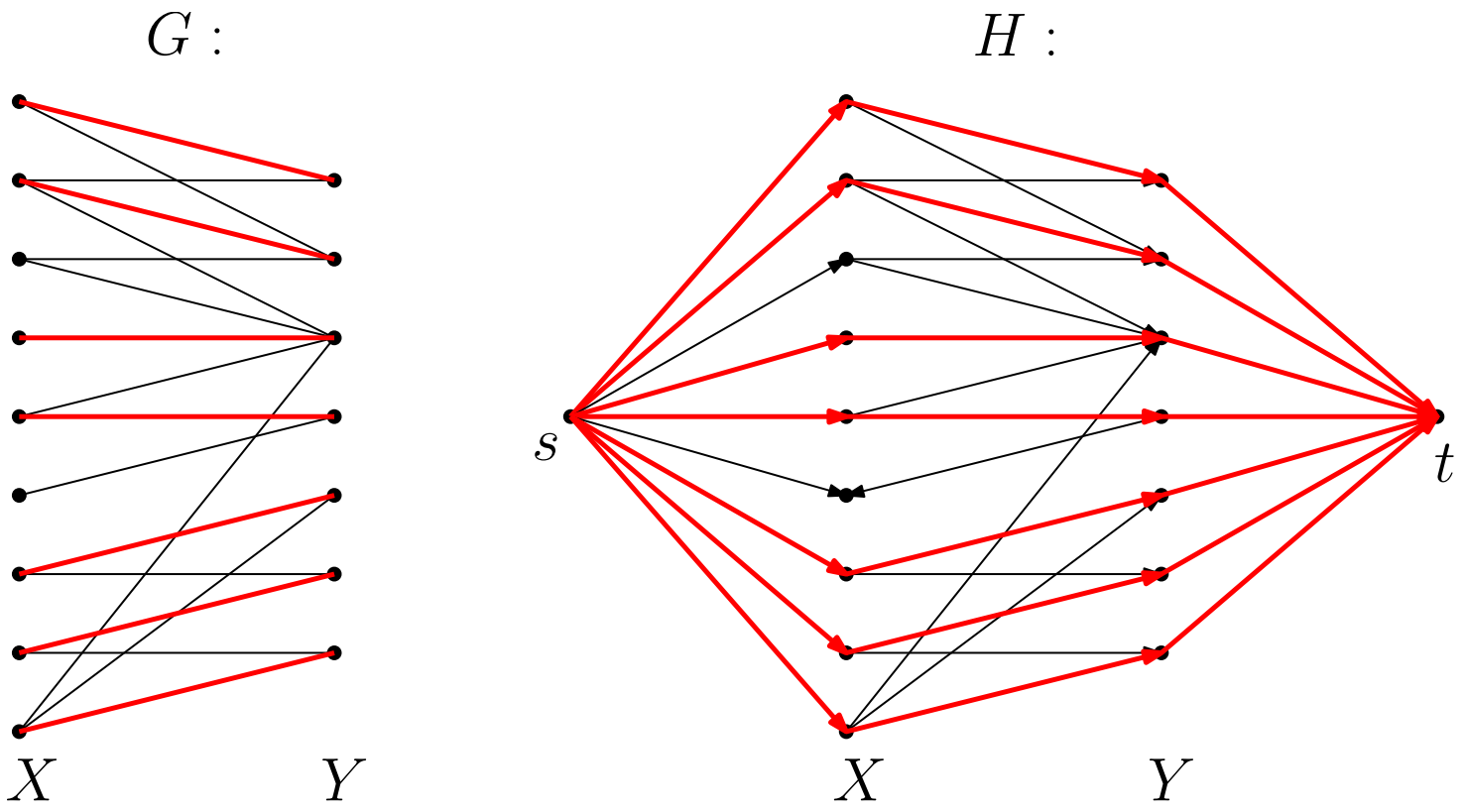
Maximum Bipartite Matching

1. we include in H all vertices of G, plus a new source s and sink t
2. we add all edges of G and direct them all from X to Y
3. we add a directed edge from s to all vertices in X, and from all vertices of Y to t
4. we assign capacity 1 to all edges of H



Maximum Bipartite Matching

1. we include in H all vertices of G, plus a new source s and sink t
2. we add all edges of G and direct them all from X to Y
3. we add a directed edge from s to all vertices in X, and from all vertices of Y to t
4. we assign capacity 1 to all edges of H



Maximum Bipartite Matching

Given a maximum flow f in H:

1. we can safely assume that the flow on each edge is 0 or 1
2. we define a set M of edges in G such that $f(e) = 1 \Rightarrow e \in M$

Maximum Bipartite Matching

Given a maximum flow f in H:

1. we can safely assume that the flow on each edge is 0 or 1
2. we define a set M of edges in G such that $f(e) = 1 \Rightarrow e \in M$
3. each $x \in X$ has exactly one incoming edge $\Rightarrow$ at most one outgoing edge of x has non-zero flow in f [by the conservation rule]
4. similarly, each $y \in Y$ has exactly one outgoing edge $\Rightarrow$ at most one incoming edge has non-zero flow

Maximum Bipartite Matching

Given a maximum flow f in H:

1. we can safely assume that the flow on each edge is 0 or 1
2. we define a set M of edges in G such that $f(e) = 1 \Rightarrow e \in M$
3. each $x \in X$ has exactly one incoming edge $\Rightarrow$ at most one outgoing edge of x has non-zero flow in f [by the conservation rule]
4. similarly, each $y \in Y$ has exactly one outgoing edge $\Rightarrow$ at most one incoming edge has non-zero flow

$\Rightarrow$ each vertex in X is paired by M with at most one vertex of Y and vice versa

$\Rightarrow$ M is a matching, where $|M| = |f|$

Maximum Bipartite Matching

Conversely, given a maximum matching M in G:

1. for each edge e of H that also belongs to G:
   - if $e \in M$, we set $f(e) = 1$,
   - otherwise we set $f(e) = 0$

Maximum Bipartite Matching

Conversely, given a maximum matching M in G:

1. for each edge e of H that also belongs to G:
   - if $e \in M$, we set $f(e) = 1$,
   - otherwise we set $f(e) = 0$
2. for each edge e incident to s or t:
   - if e shares an endpoint with an edge of M, we set $f(e) = 1$,
   - otherwise we set $f(e) = 0$

Maximum Bipartite Matching

Conversely, given a maximum matching M in G:

1. for each edge e of H that also belongs to G:
   - if $e \in M$, we set $f(e) = 1$,
   - otherwise we set $f(e) = 0$
2. for each edge e incident to s or t:
   - if e shares an endpoint with an edge of M, we set $f(e) = 1$,
   - otherwise we set $f(e) = 0$
3. it is easy to verify that:
   - f is a flow in H [satisfies all three conditions]
   - the value of f is $|f| = |M|$

Maximum Bipartite Matching

Conversely, given a maximum matching M in G:

1. for each edge e of H that also belongs to G:
   - if $e \in M$, we set $f(e) = 1$,
   - otherwise we set $f(e) = 0$
2. for each edge e incident to s or t:
   - if e shares an endpoint with an edge of M, we set $f(e) = 1$,
   - otherwise we set $f(e) = 0$
3. it is easy to verify that:
   - f is a flow in H [satisfies all three conditions]
   - the value of f is $|f| = |M|$

Therefore: maximum matching in G = maximum flow in H

Maximum Bipartite Matching: running time

Given a bipartite graph G with n vertices and m edges:

- we construct H from G in $O(n+m)$ time
- we compute a maximum flow in H using (the standard) Ford-Fulkerson algorithm
- the value of the maximum flow is $|f| = |M| \leq \frac{n}{2}$

$\Rightarrow$ at most $\frac{n}{2}$ flow augmentations

$\Rightarrow$ running time: $O(n \cdot (n+m)) = O(n \cdot m)$

- finally construction of M from f in $O(n+m)$ time

Maximum Bipartite Matching: running time

Given a bipartite graph G with n vertices and m edges:

- we construct H from G in $O(n+m)$ time
- we compute a maximum flow in H using (the standard) Ford-Fulkerson algorithm
- the value of the maximum flow is $|f| = |M| \leq \frac{n}{2}$

$\Rightarrow$ at most $\frac{n}{2}$ flow augmentations

$\Rightarrow$ running time: $O(n \cdot (n+m)) = O(n \cdot m)$

- finally construction of M from f in $O(n+m)$ time

The fastest known algorithm has running time: $O(m \cdot \sqrt{n})$
(by simultaneously augmenting many paths)

Additional slides

Proof of the "Net flow" lemma.

Note $V = T \cup S$.

$$\begin{aligned} f(S,T) &= f(S,V) - f(S,S) = f(S,V) - 0 \\ &= f(s,V) + f(S-s,V) = f(s,V) + 0 \quad \text{(why???)} \\ &= |f| \text{ by def.} \end{aligned}$$

$\square$

Proof of max-flow min-cut theorem.

(1) $\Longrightarrow$ **(2)** [f max flow $\Rightarrow$ no augmenting path]

Proof of max-flow min-cut theorem.

(1) $\Longrightarrow$ **(2)** [f max flow $\Rightarrow$ no augmenting path]
Suppose that f is a maximum flow in G but G_f has an augmenting path p. Then by earlier corollary "Improve", flow sum $f + f_p$ is flow in G w/ value strictly greater than $|f|$, contradiction!

Proof of max-flow min-cut theorem.

(1) $\Longrightarrow$ **(2)** [f max flow $\Rightarrow$ no augmenting path]
Suppose that f is a maximum flow in G but G_f has an augmenting path p. Then by earlier corollary "Improve", flow sum $f + f_p$ is flow in G w/ value strictly greater than $|f|$, contradiction!

(2) $\Longrightarrow$ **(3)** [no augm. path $\Rightarrow |f| = c(S, T)$ for some cut (S, T)]

Proof of max-flow min-cut theorem.

(1) $\Longrightarrow$ (2) [f max flow $\Rightarrow$ no augmenting path]
Suppose that f is a maximum flow in G but G_f has an augmenting path p. Then by earlier corollary "Improve", flow sum $f + f_p$ is flow in G w/ value strictly greater than $|f|$, contradiction!

(2) $\Longrightarrow$ (3) [no augm. path $\Rightarrow |f| = c(S, T)$ for some cut (S, T)]
Suppose G_f doesn't have augmenting path, i.e., G_f contains no path from s to t. Define

$$S = \{v \in V : \exists \text{ path from } s \text{ to } v \text{ in } G_f\} \quad \text{and} \quad T = V - S$$

Partition (S, T) is a cut because $s \in S$ and $t \notin S$. For each $u \in S, v \in T$ we have $f(u, v) = c(u, v)$, because otherwise $(u, v) \in E_f$, v would be in S ($s \rightsquigarrow u \to v$). By earlier Lemma "Net flow", $|f| = f(S, T) = c(S, T)$.

Proof of max-flow min-cut theorem.

(1) $\Longrightarrow$ (2) [f max flow $\Rightarrow$ no augmenting path]
Suppose that f is a maximum flow in G but G_f has an augmenting path p. Then by earlier corollary "Improve", flow sum $f + f_p$ is flow in G w/ value strictly greater than $|f|$, contradiction!

(2) $\Longrightarrow$ (3) [no augm. path $\Rightarrow |f| = c(S, T)$ for some cut (S, T)]
Suppose G_f doesn't have augmenting path, i.e., G_f contains no path from s to t. Define

$$S = \{v \in V : \exists \text{ path from } s \text{ to } v \text{ in } G_f\} \quad \text{and} \quad T = V - S$$

Partition (S, T) is a cut because $s \in S$ and $t \notin S$. For each $u \in S, v \in T$ we have $f(u, v) = c(u, v)$, because otherwise $(u, v) \in E_f$, v would be in S ($s \rightsquigarrow u \to v$). By earlier Lemma "Net flow", $|f| = f(S, T) = c(S, T)$.

(3) $\Longrightarrow$ (1) [$|f| = c(S, T)$ for some cut $(S, T) \Rightarrow f$ max flow]

Proof of max-flow min-cut theorem.

(1) $\Longrightarrow$ (2) [f max flow $\Rightarrow$ no augmenting path]
Suppose that f is a maximum flow in G but G_f has an augmenting path p. Then by earlier corollary "Improve", flow sum $f + f_p$ is flow in G w/ value strictly greater than $|f|$, contradiction!

(2) $\Longrightarrow$ (3) [no augm. path $\Rightarrow |f| = c(S, T)$ for some cut (S, T)]
Suppose G_f doesn't have augmenting path, i.e., G_f contains no path from s to t. Define

$$S = \{v \in V : \exists \text{ path from } s \text{ to } v \text{ in } G_f\} \quad \text{and} \quad T = V - S$$

Partition (S, T) is a cut because $s \in S$ and $t \notin S$. For each $u \in S, v \in T$ we have $f(u, v) = c(u, v)$, because otherwise $(u, v) \in E_f$, v would be in S ($s \rightsquigarrow u \to v$). By earlier Lemma "Net flow", $|f| = f(S, T) = c(S, T)$.

(3) $\Longrightarrow$ (1) [$|f| = c(S, T)$ for some cut $(S, T) \Rightarrow f$ max flow]
By earlier corollary "Upper cut", $|f| \leq c(S, T)$ for all cuts (S, T).
Condition $|f| = c(S, T)$ implies that f is maximum flow. $\square$